

```
#[opcode(7'b0101011)]
#[funct3(0x3)]
rtype stream_add
(rs1,rs2,funct7)->(rd){
  let s = rf[rs1];
  let d = rf[rs2];
  let n = funct7;
  for(var i=0;i<n;i=i+1){
    mem[d]=mem[s]+mem[s+4];
    s += 8; d += 4;
  }
}
```

```
stream_add: {
  identification:
    inst[6:0]==7'b0101011
    && inst[14:12]==0x4
  decode: {
    rs1=inst[19:15]
    rs2=inst[24:20]
    funct7=inst[31:25]
  }
  behavior: {
    CDFG: { .. }
  }
}
```

```
init: {
  s=RegRdResp(RegRdReq(rs1))
  d=RegRdResp(RegRdReq(rs2))
  n=funct7; i=0; cond=0<n
}
loop: {
  v0=MemRdResp(MemRdReq(s))
  v1=MemRdResp(MemRdReq(s+4))
  MemWrReq(d, v0+v1)
  s=s+8; d=d+4
  i=i+1; cond=i+1<n
}
done: {}
```

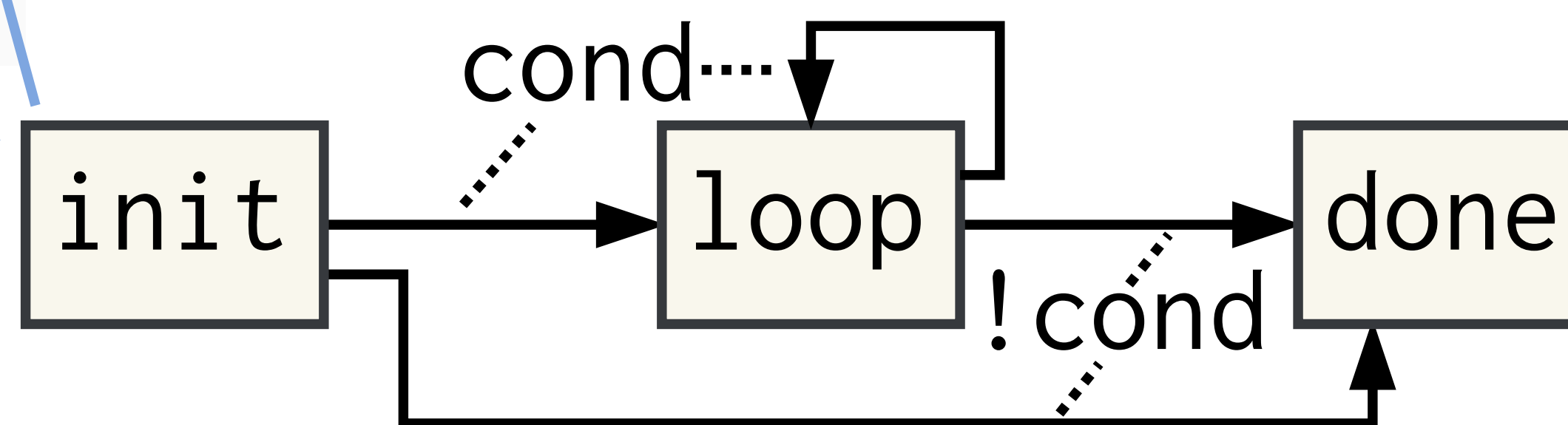
```
loop: {
  II: 2,
  s0: t0=MemRdReq(s)
      s'=s; s=s+8; i=i+1
      t0'=t0; cond=i+1<n
  s1: t1=MemRdReq(s'+4)
      v0=MemRdResp(t0')
      t1'=t1; v0'=v0;
  s2: v1=MemRdResp(t1')
      MemWrReq(d,v0'+v1)
      d=d+4
}
```

```
FSM[true] {}
init[idle&enable] {}
s0[c0] {}
s1[c1] {}
s2[c2] {
  v1=MemRdResp(t1')
  MemWrReq(d,v0'+v1)
  d=d+4
}
```

(a) CADL Input

Parse

(b) HIR



Schedule

(c) LIR

Implementation

(d) RuleIR → RTL

μarchitecture-agnostic

μarchitecture-specific